CS851: Network Security

Assignment 3 Lab Report

Submitted by

SAMYAK SHRIRAM GEDAM

Roll No: 252IS032

II Semester M-Tech CSIS

Department of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore-575025, India

2025–2026

Attack on ECB (Electronic Codebook) Mode

Aim:
To implement AES encryption using ECB mode and demonstrate its weakness by showing that identical plaintext blocks produce identical ciphertext blocks, thereby proving that ECB is vulnerable to pattern leakage attacks

Algorithm Used:
Advanced Encryption Standard (AES-128)

Tool Used:
OpenSSL on Linux

Key Used:
`00112233445566778899aabbccddeeff`

Description:
Electronic Codebook (ECB) mode is the simplest block cipher mode where each plaintext block is encrypted independently using the same key. Since ECB does not use an Initialization Vector (IV) or chaining,identical plaintext blocks always generate identical ciphertext blocks. This property allows attackers to detect patterns in encrypted data without knowing the key

Attack Strategy:
An attacker deliberately chooses a plaintext consisting of repeated 16-byte blocks (AES block size). This input exploits the independent block encryption property of ECB mode and reveals patterns directly in the ciphertext.

Procedure:

1. A plaintext file containing repeated characters (`A`) was created such that each 16-byte block is identical.
2. The plaintext was encrypted using AES-128 in ECB mode with a fixed secret key.
3. The ciphertext output was examined using a hexadecimal dump tool.
4. The plaintext was modified by changing only one block and the encryption was repeated.

Observation:
It was observed that identical plaintext blocks resulted in identical ciphertext blocks. ECB provides no randomness or diffusion, When only one plaintext block was modified, only the corresponding ciphertext block changed, while other blocks remained unchanged. This clearly reveals the structure present in the plaintext.

Result:
The experiment successfully demonstrated that ECB mode fails to hide plaintext patterns. The ciphertext directly reflects repeated data, proving that ECB does not provide semantic security.

Conclusion:
ECB mode is insecure for encrypting structured or repetitive data. Due to its deterministic nature and lack of inter-block dependency, ECB is vulnerable to pattern analysis attacks and should not be used in real-world cryptographic applications.

Screenshots:

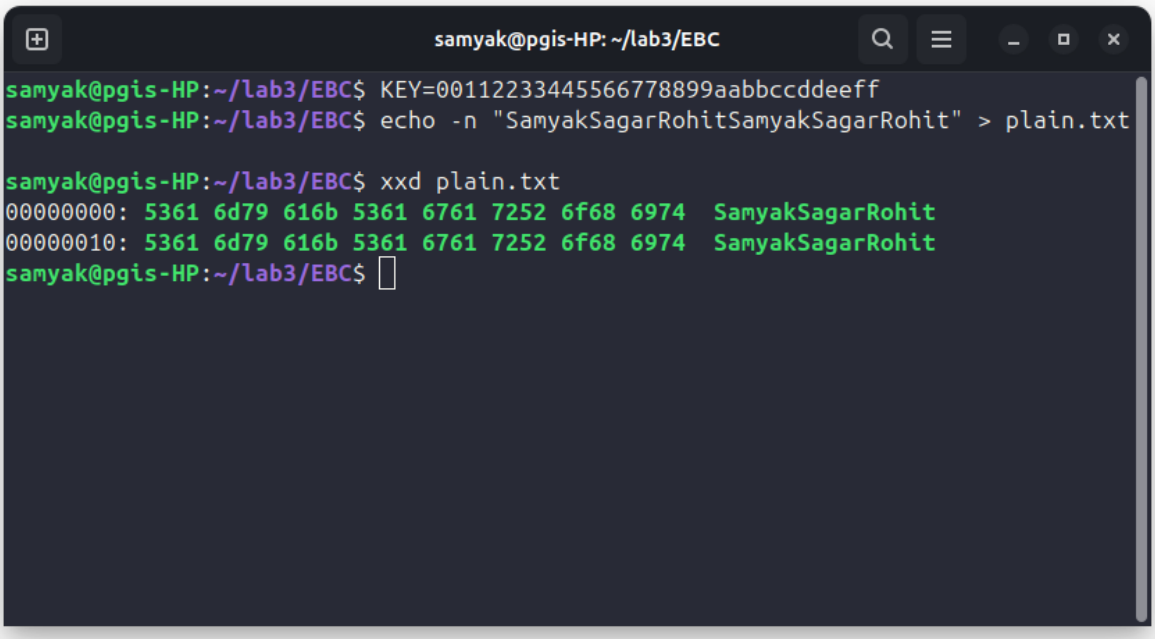

Figure 1: Creation of Repetitive Plaintext for ECB Attack

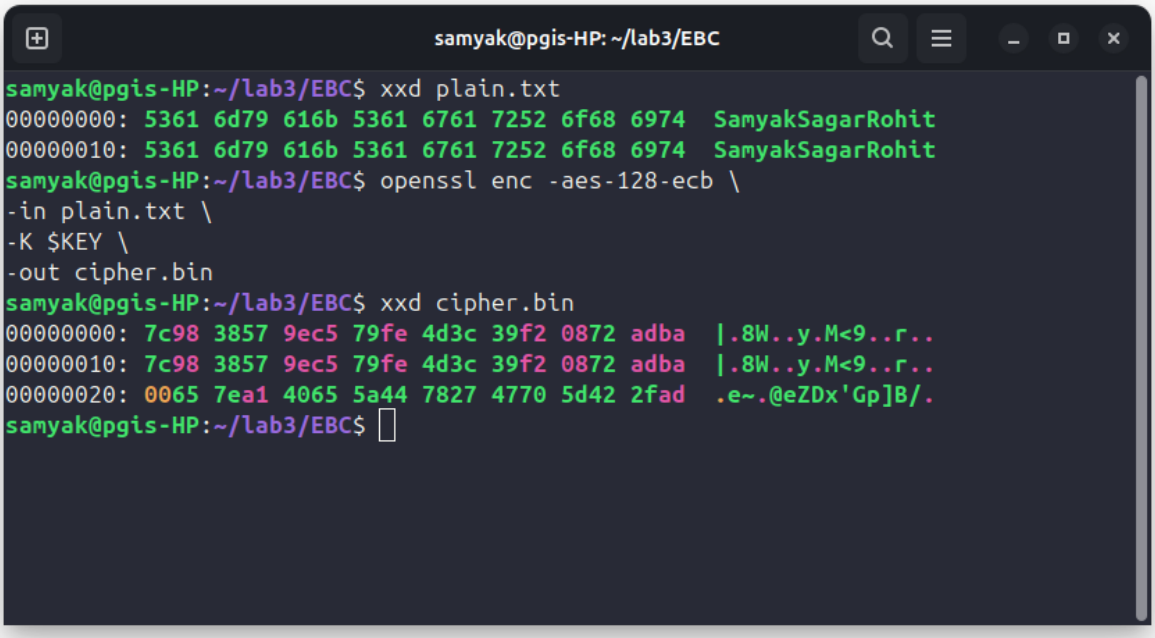

Figure 2: AES-128 Encryption Using ECB Mode

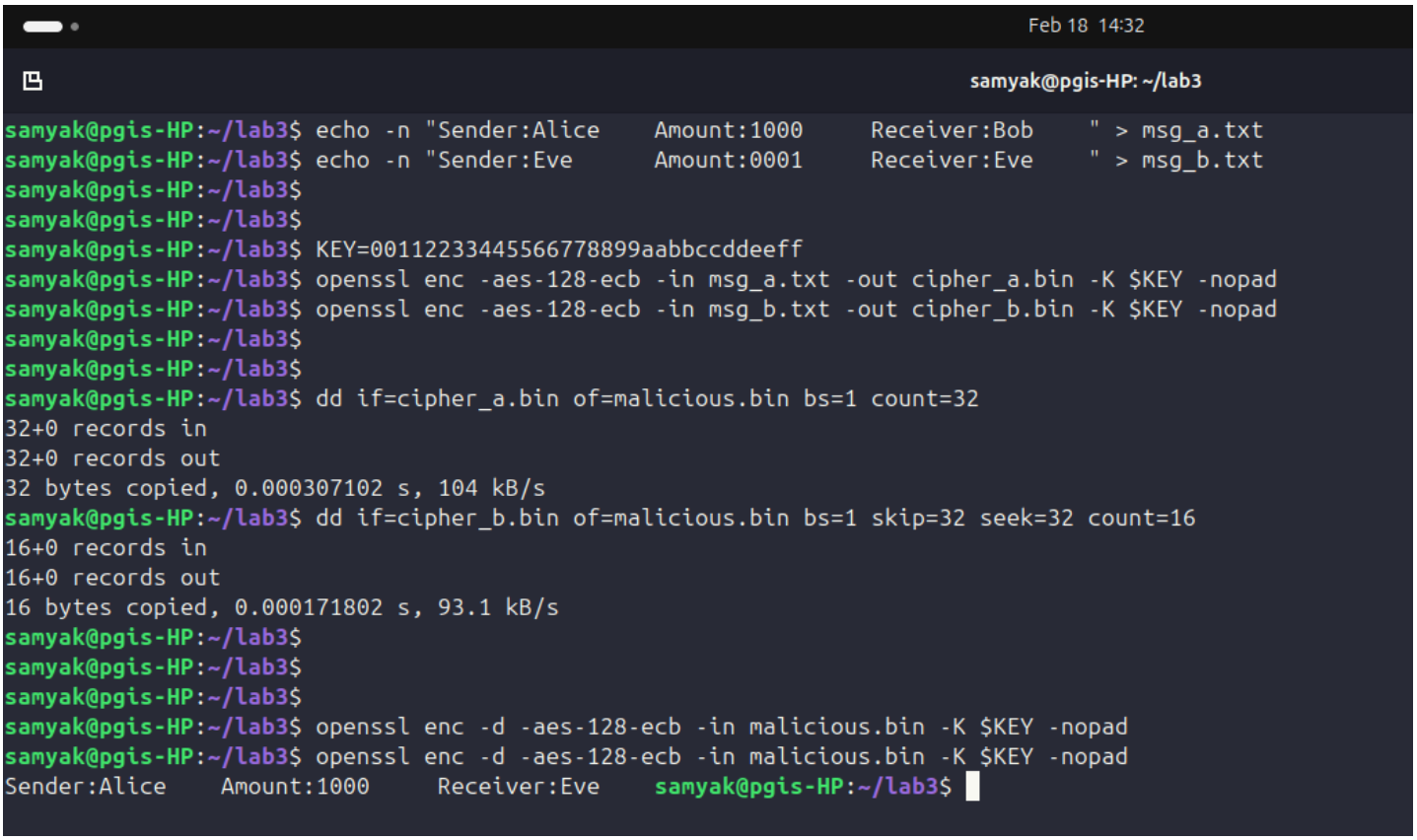

Figure 3: ECB Cut-and-Paste Attack

Attack Demonstrated: Pattern Leakage / Block Replay Attack
An attacker observing the ciphertext can:

- Detect repeated patterns
- Infer structure of the plaintext
- Identify repeating data (e.g., images, formats)

This attack is possible without knowing the encryption key.

Attack on CBC (Cipher Block Chaining) Mode

Aim:
To implement AES encryption using CBC mode and demonstrate its weakness by performing a bit-flipping attack, proving that CBC mode is vulnerable to ciphertext manipulation and does not provide data integrity.

Algorithm Used:
Advanced Encryption Standard (AES-128)

Tool Used:
OpenSSL on Linux

Key Used:
`00112233445566778899aabbccddeeff`

Initialization Vector (IV) Used:
`0102030405060708090a0b0c0d0e0f10`

Description:
Cipher Block Chaining (CBC) mode encrypts each plaintext block by XORing it with the previous ciphertext block before encryption. The first block uses an Initialization Vector (IV). While CBC successfully hides plaintext patterns, it does not provide integrity protection. Any modification in the ciphertext directly affects the decrypted plaintext in a predictable manner.

Attack Strategy:
An attacker modifies one or more bits of the ciphertext without knowing the encryption key. Due to the XOR operation during decryption, flipping bits in a ciphertext block causes controlled corruption in the corresponding decrypted plaintext block. This demonstrates the malleability of CBC mode.

Procedure:

1. A structured plaintext containing identifiable fields such as user role and user ID was created.
2. The plaintext was encrypted using AES-128 in CBC mode with a fixed key and IV.
3. One byte of the ciphertext was deliberately modified using a binary editing command.
4. The modified ciphertext was decrypted using the same key and IV.
5. The decrypted output was analyzed to observe changes in the plaintext.

Observation:
It was observed that modifying a single byte of ciphertext resulted in corruption of the decrypted plaintext. Part of the plaintext was altered or truncated without any authentication failure. Although a padding-related warning was generated, the decrypted data still revealed modified plaintext content.

Result:
The experiment demonstrated that CBC mode is vulnerable to bit-flipping attacks. An attacker can manipulate ciphertext to alter the decrypted plaintext without knowing the encryption key, proving that CBC does not ensure data integrity.

Conclusion:
While CBC mode successfully hides plaintext patterns and improves confidentiality over ECB mode, it does not provide protection against ciphertext modification. Due to its malleability, CBC must be combined with an authentication mechanism such as a Message Authentication Code (MAC) to be secure in practical applications.

Screenshots:

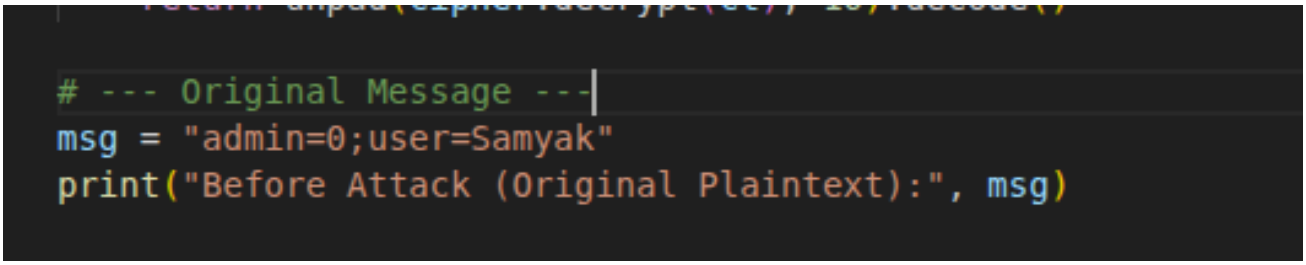

Figure 4: Structured Plaintext for CBC Attack

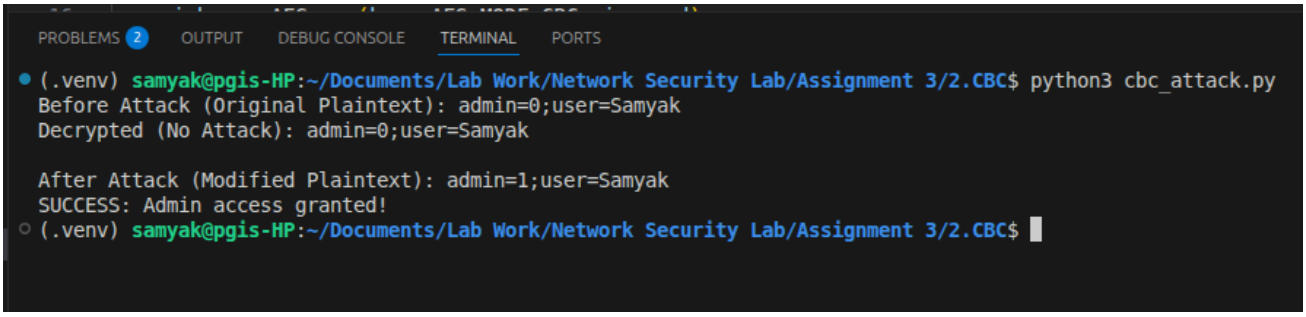

Figure 5: After Attack

Attack Demonstrated: Bit-Flipping / Ciphertext Malleability Attack

An attacker observing or intercepting the ciphertext can:

- Modify ciphertext blocks without knowing the encryption key
- Cause predictable changes in decrypted plaintext
- Bypass integrity checks in unauthenticated CBC encryption

This attack demonstrates that CBC mode does not provide integrity or authenticity by itself.

Attack on CFB (Cipher Feedback) Mode

Aim:
To implement AES encryption using CFB mode and demonstrate its weakness by performing a controlled bit-flipping attack, proving that CFB mode is vulnerable to ciphertext manipulation and does not provide data integrity.

Algorithm Used:
Advanced Encryption Standard (AES-128)

Tool Used:
OpenSSL on Linux

Key Used:
`00112233445566778899aabbccddeeff`

Initialization Vector (IV) Used:
`0102030405060708090a0b0c0d0e0f10`

Description:
Cipher Feedback (CFB) mode converts a block cipher into a stream cipher. In CFB mode, plaintext is XORed with a keystream generated from the encryption of the previous ciphertext block (or IV for the first block). While CFB mode hides plaintext patterns effectively, it does not provide integrity protection. Since encryption and decryption rely on XOR operations, modification of ciphertext directly results in predictable changes in the decrypted plaintext.

Attack Strategy:
An attacker modifies selected bytes of the ciphertext by computing the XOR difference between the original plaintext and the desired plaintext. Because CFB operates as a stream cipher, altering ciphertext bytes results in controlled modification of the decrypted plaintext without requiring knowledge of the encryption key.

Procedure:

1. A plaintext message containing numeric values was created (e.g., `send me 1000`).
2. The plaintext was encrypted using AES-128 in CFB mode with a fixed key and IV.
3. The XOR difference between `1000` and the desired value `9999` was computed.
4. The corresponding ciphertext bytes were modified using the calculated XOR values.
5. The modified ciphertext was decrypted using the same key and IV.

Observation:
It was observed that modifying selected ciphertext bytes resulted in precise and controlled changes in the decrypted plaintext. The value `1000` was successfully altered to `9999` without generating any decryption or padding errors. The attack occurred without knowledge of the secret key.

Result:
The experiment demonstrated that CFB mode is vulnerable to bit-flipping attacks. Since CFB behaves like a stream cipher, ciphertext modification directly affects plaintext, proving that CFB mode does not ensure data integrity.

Conclusion:
CFB mode provides confidentiality by hiding plaintext patterns; however, it does not protect against ciphertext tampering. Due to its malleability, CFB must be combined with an authentication mechanism such as a Message Authentication Code (MAC) to be secure in practical applications.

Screenshots:

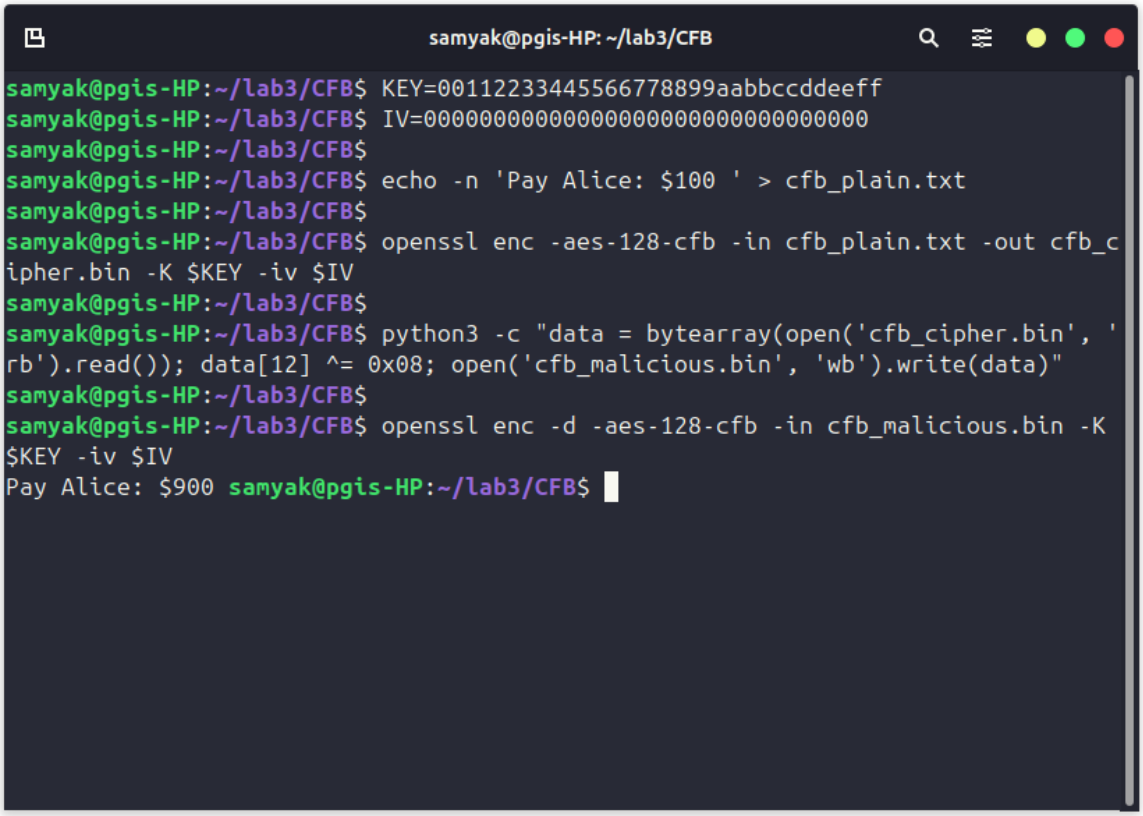

Figure 6: Bit-Flipping (Malleability) Attack

Attack Demonstrated: Byte-Level Bit-Flipping / Stream Cipher Malleability Attack
An attacker intercepting the ciphertext can:

- Modify specific ciphertext bytes without knowing the encryption key
- Cause predictable and controlled changes in decrypted plaintext
- Manipulate numeric or structured data undetected in unauthenticated CFB encryption

This attack demonstrates that CFB mode does not provide integrity or authenticity by itself.

Attack on OFB (Output Feedback) Mode – Keystream Reuse

Aim:
To implement AES encryption using OFB mode and demonstrate that reusing the same key and IV (Initialization Vector) results in keystream reuse, leading to complete loss of confidentiality.

Algorithm Used:
Advanced Encryption Standard (AES-128)

Tool Used:
OpenSSL and Python on Linux

Key Used:
`00112233445566778899aabbccddeeff`

Initialization Vector (IV) Used:
`00000000000000000000000000000000`

Description:
Output Feedback (OFB) mode converts a block cipher into a synchronous stream cipher by generating a keystream independent of plaintext and ciphertext. Encryption is performed by XORing plaintext with the generated keystream. If the same key and IV are reused, the identical keystream is produced again, resulting in a critical vulnerability known as keystream reuse.

Attack Strategy:
Two different plaintext messages were encrypted using the same key and IV in OFB mode. Since the same keystream was generated for both encryptions, the attacker used a known plaintext-ciphertext pair to recover the keystream. The recovered keystream was then used to decrypt the second (secret) ciphertext without knowledge of the encryption key.

Procedure:

1. Two plaintext messages were created: one known to the attacker and one secret.
2. Both plaintexts were encrypted using AES-128-OFB with the same key and IV.
3. The ciphertext corresponding to the known plaintext was XORed with its plaintext to recover the keystream.
4. The recovered keystream was XORed with the second ciphertext.
5. The second (secret) plaintext was successfully recovered.

Observation:
It was observed that using the same key and IV in OFB mode generated the same keystream for both messages. By XORing the known plaintext with its ciphertext, the attacker successfully recovered the keystream. Using this recovered keystream, the second encrypted message was fully decrypted without knowledge of the secret key.

Result:
The experiment demonstrated that IV reuse in OFB mode results in keystream reuse, which leads to complete compromise of encrypted messages. If one plaintext-ciphertext pair is known, all other ciphertexts encrypted with the same key and IV can be decrypted.

Conclusion:
OFB mode provides strong confidentiality only if the IV is unique for every encryption. Reusing the same key and IV results in identical keystream generation, allowing attackers to recover plaintexts easily. Therefore, strict IV management is essential when using OFB mode in practical cryptographic systems.

Screenshots:

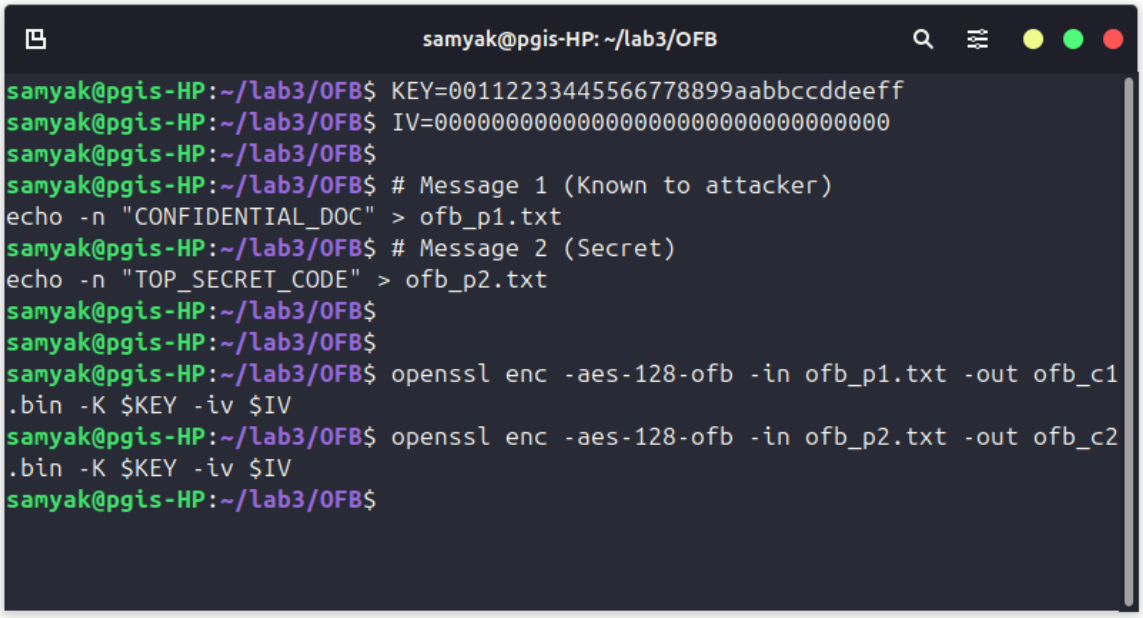

Figure 7: Encryption of Two Messages Using Same Key and IV in OFB Mode

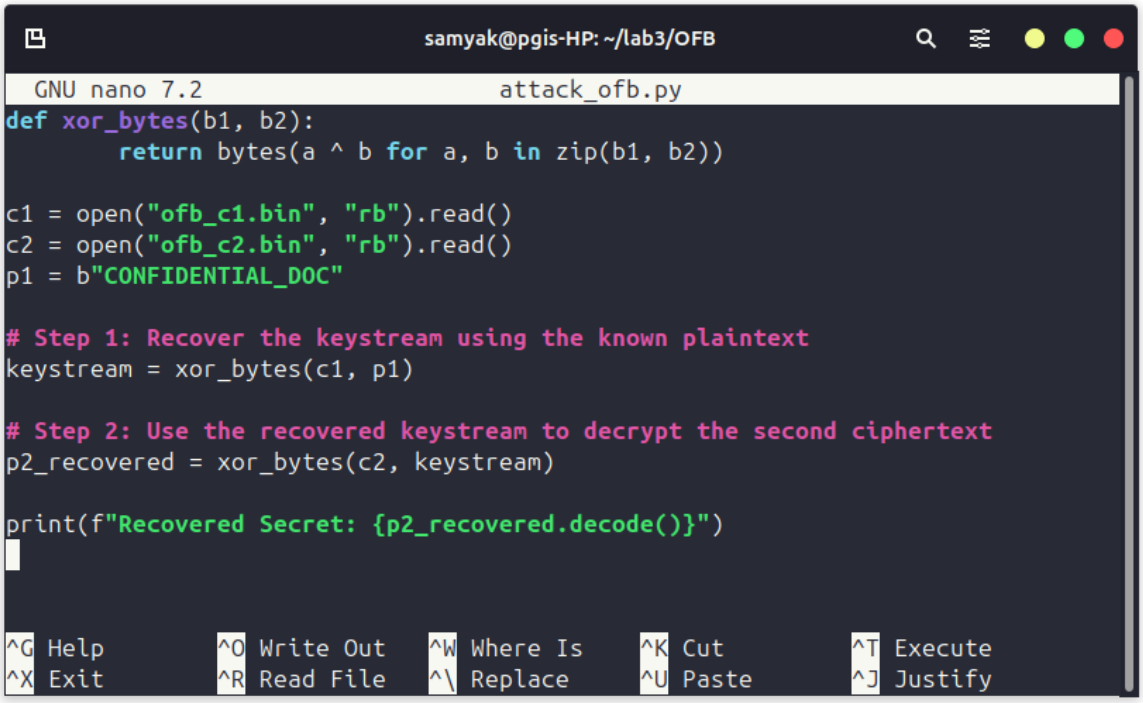

Figure 8: Python Script Used to Recover Keystream by XOR Operation

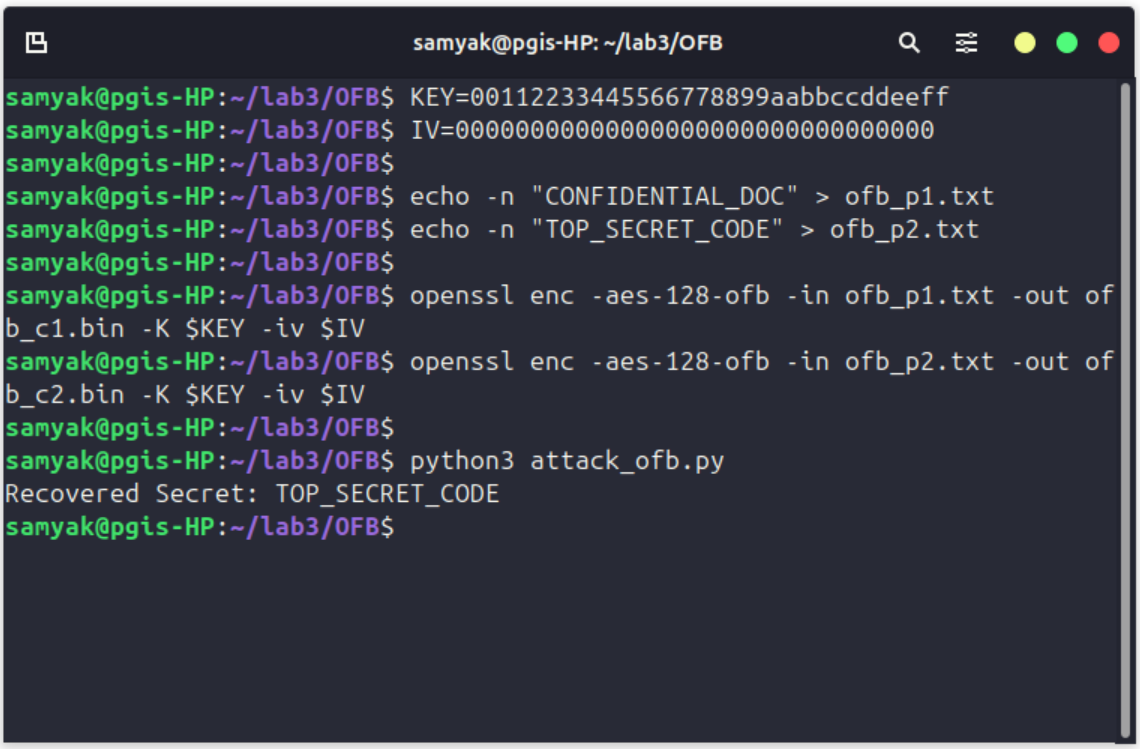

Figure 9: Successful Recovery of Secret Plaintext Using Recovered Keystream

Attack Demonstrated: Keystream Reuse / IV Reuse Attack

An attacker observing ciphertexts encrypted with the same key and IV can:

- Recover the keystream using a known plaintext-ciphertext pair
- Decrypt other ciphertexts encrypted with the same IV
- Completely break confidentiality without knowing the secret key

This attack demonstrates that OFB mode becomes insecure when the IV is reused.

Attack on CTR (Counter) Mode

Aim:
To implement AES encryption using CTR mode and demonstrate its vulnerability when the same key and nonce (IV) are reused, proving that CTR mode fails catastrophically under nonce reuse.

Algorithm Used:
Advanced Encryption Standard (AES-128)

Tool Used:
OpenSSL on Linux

Key Used:
`00112233445566778899aabbccddeeff`

Initialization Vector (Nonce) Used:
`0102030405060708090a0b0c0d0e0f10`

Description:
Counter (CTR) mode converts a block cipher into a stream cipher by generating a keystream using encrypted counter values combined with a nonce (IV). The plaintext is XORed with this keystream to produce ciphertext. CTR mode provides strong confidentiality only if the same key and nonce pair are never reused. If nonce reuse occurs, the same keystream is generated, resulting in severe security compromise.

Attack Strategy:
Two different plaintext messages of equal length were encrypted using the same key and the same nonce in CTR mode. Since CTR generates identical keystreams under nonce reuse, XORing the two ciphertexts eliminates the keystream and reveals the XOR of the two plaintexts. If one plaintext is known, the second plaintext can be fully recovered.

Procedure:

1. Two equal-length plaintext messages were created.
2. Both messages were encrypted using AES-128 in CTR mode with the same key and nonce.
3. The two resulting ciphertexts were XORed together using a Python script.
4. It was verified that the result equals the XOR of the two original plaintexts.
5. Using one known plaintext, the second plaintext was successfully recovered.

Observation:
It was observed that XORing the two ciphertexts produced the XOR of the original plaintexts, confirming that the same keystream was reused. When one plaintext was known, the second plaintext was successfully recovered without knowing the secret key. This demonstrates complete loss of confidentiality under nonce reuse.

Result:
The experiment proved that CTR mode becomes insecure if the same key and nonce pair are reused. Keystream reuse allows attackers to derive relationships between plaintexts and recover confidential information.

Conclusion:
CTR mode provides strong confidentiality only when nonce values are unique for each encryption operation. Reusing a nonce with the same key results in keystream reuse and total compromise of encrypted data. Therefore, strict nonce management is essential when using CTR mode.

Screenshots:

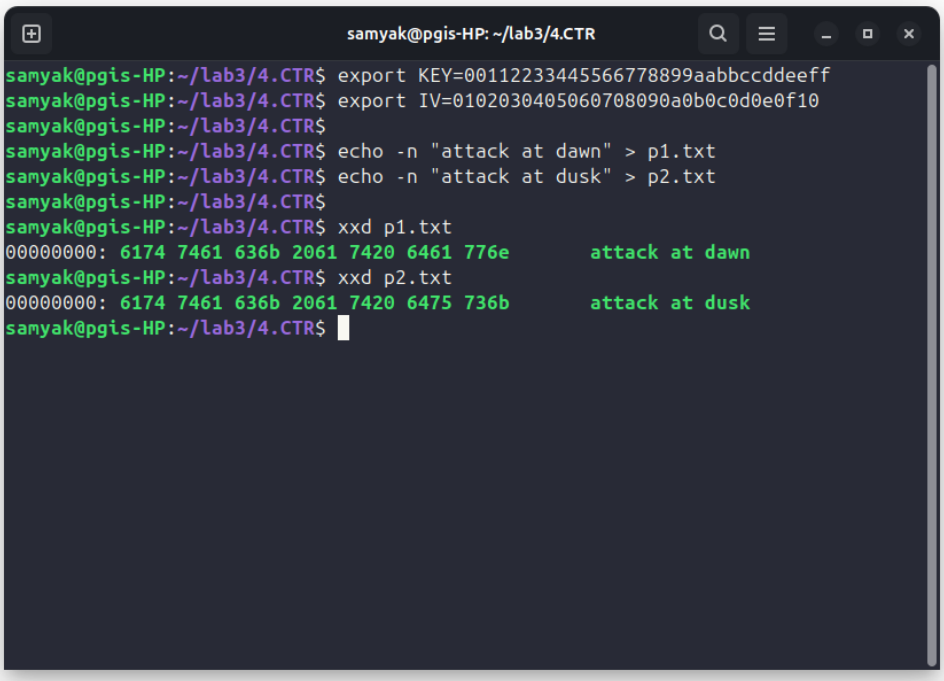

Figure 10: Creation of Two Equal-Length Plaintexts

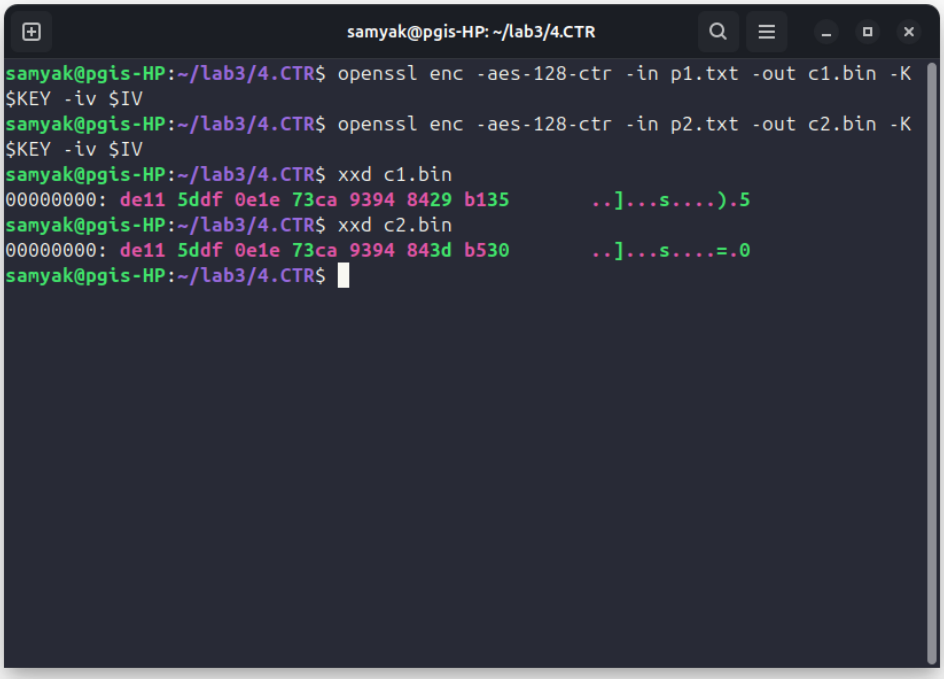

Figure 11: AES-128 Encryption of Both Plaintexts Using Same Nonce

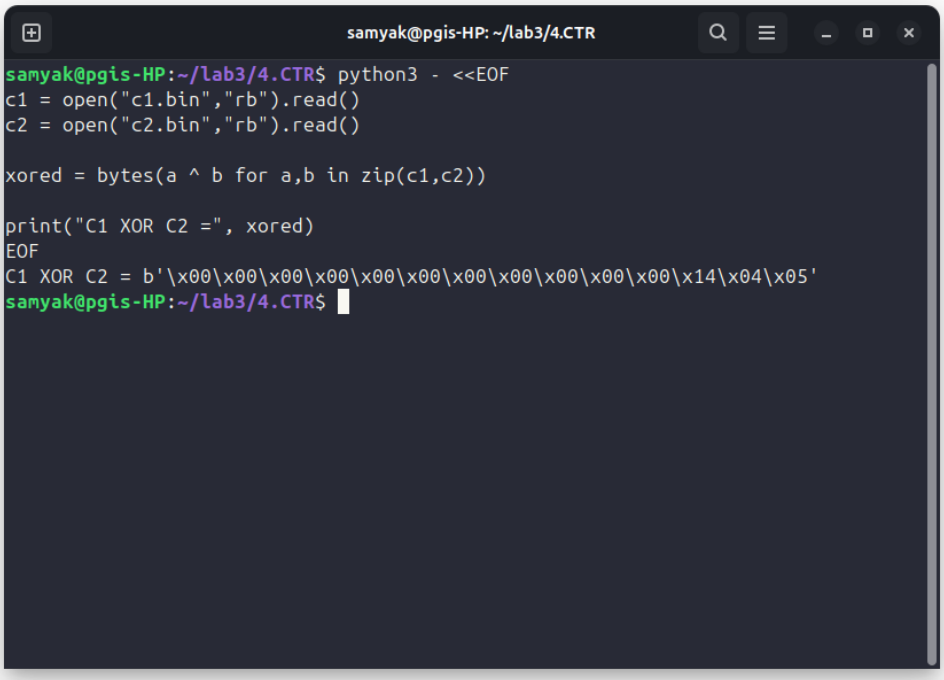

Figure 12: XOR of Ciphertexts Showing XOR of Plaintexts

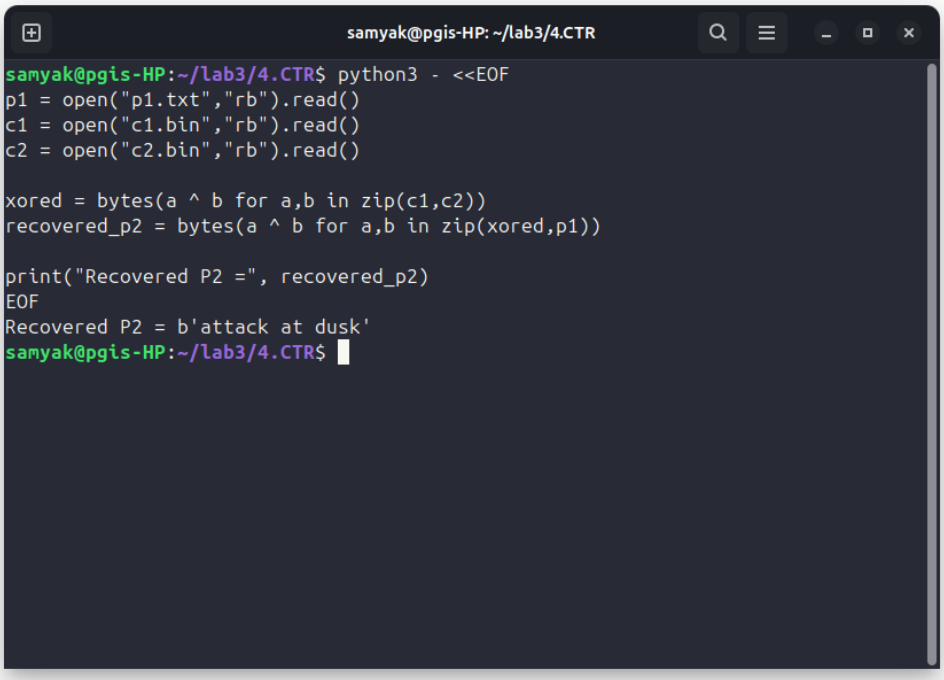

Figure 13: Recovery of Unknown Plaintext Using Known Plaintext

Attack Demonstrated: Nonce Reuse / Keystream Reuse Attack

An attacker observing ciphertexts can:

- XOR ciphertexts encrypted with the same key and nonce
- Eliminate the keystream and obtain the XOR of plaintexts
- Recover unknown plaintext if one plaintext is known
- Completely break confidentiality under nonce reuse

This attack demonstrates that CTR mode is secure only when nonce values are never reused.